

# **Cryptographic Tools-2**

## -for Confidentiality and Integrity

Dr. Yiwen Hu

# Announcement

- **HW1 will be released on Blackboard this Friday (09/04) by 12:00 PM**
- **The assignment is due on Wednesday, September 16, 11:59 PM (UMBC time)**
- **HW1 will include true/false, multiple-choice, and short-answer questions, and the workload should be manageable.**
- **You will have approximately 11 days to complete HW1**

# Question

## 1. Which TWO of the following are requirements for the secure use of symmetric encryption?

- **A)** The sender and receiver must obtain copies of the secret key in a secure fashion and must keep the key secret.
- **B)** The key must be made public so that anyone can verify the integrity of the message.
- **C)** The key must be exactly 64 bits long to work with the AES algorithm.
- **D)** The encryption algorithm must be strong enough that an attacker who knows the algorithm and has access to one or more ciphertexts cannot recover the plaintext or the key.
- **E)** The sender must encrypt with a public key while the receiver decrypts with a private key.

# Recap

## 1. Which TWO of the following are requirements for the secure use of symmetric encryption?

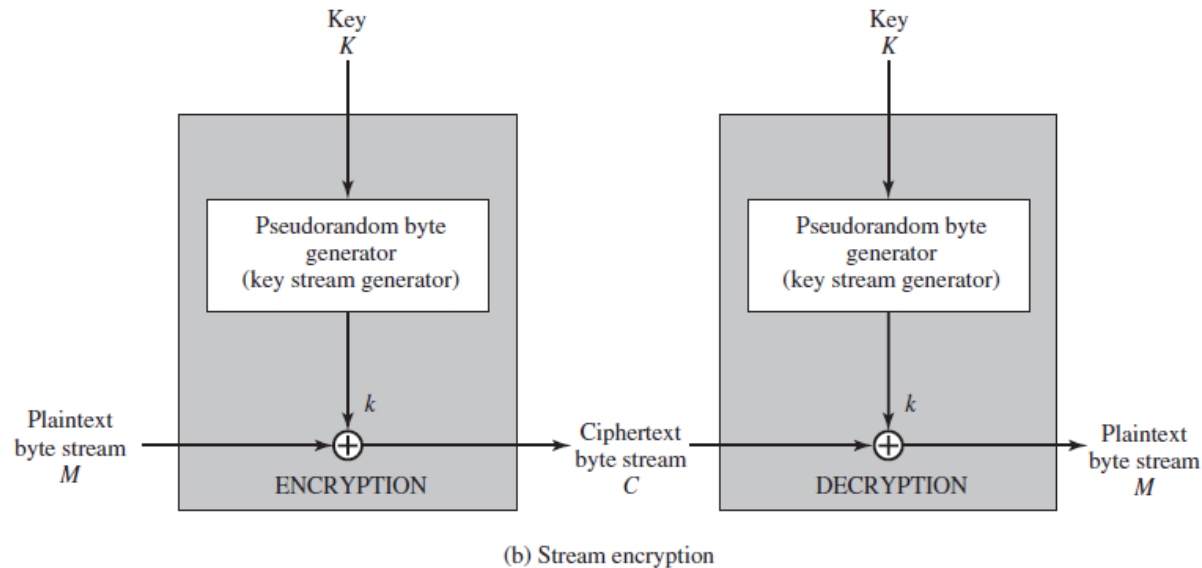
- **A)** The sender and receiver must obtain copies of the secret key in a secure fashion and must keep the key secret.
- **B)** The key must be made public so that anyone can verify the integrity of the message.
- **C)** The key must be exactly 64 bits long to work with the AES algorithm.
- **D)** The encryption algorithm must be strong enough that an attacker who knows the algorithm and has access to one or more ciphertexts cannot recover the plaintext or the key.
- **E)** The sender must encrypt with a public key while the receiver decrypts with a private key.

|                              | DES | Triple DES | AES              |
|------------------------------|-----|------------|------------------|
| Plaintext block size (bits)  | 64  | 64         | 128              |
| Ciphertext block size (bits) | 64  | 64         | 128              |
| Key size (bits)              | 56  | 112 or 168 | 128, 192, or 256 |

DES = Data Encryption Standard  
AES = Advanced Encryption Standard

# Stream cipher

- Encrypt and decrypt data one bit or byte at a time
- A keystream generator takes the secret key (and a nonce/IV) and produces a pseudorandom keystream. For each plaintext bit/byte  $M$ , one keystream bit/byte  $K_i$  is generated and XORed with it:



$$\text{Ciphertext } C = M \oplus K_i$$

$$\text{Decryption uses the same keystream: } M = C \oplus K_i$$



| A | B | Q |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

|                                   | Block Cipher                                                | Stream Cipher                                                                                                      |
|-----------------------------------|-------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>How it works</b>               | Encrypts fixed-size blocks of plaintext one block at a time | Generates a pseudorandom keystream from the key (+ nonce/IV) and XORs it with plaintext bit-by-bit or byte-by-byte |
| <b>Block / unit size</b>          | DES, 3DES: 64-bit blocks;<br>AES: 128-bit blocks            | 1 bit or 1 byte at a time                                                                                          |
| <b>Examples</b>                   | DES, 3DES, AES                                              | RC4, ChaCha20                                                                                                      |
| <b>Needs a mode of operation?</b> | Yes — ECB, CBC, CTR, GCM, etc.                              | No                                                                                                                 |
| <b>Speed / hardware</b>           | Slow; well suited to software, parallelizable in CTR/GCM    | Very fast; low memory; simple hardware                                                                             |
| <b>Typical use</b>                | Files, databases, TLS (AES-GCM), disk encryption            | Real-time streams — voice, video, wireless links, TLS (ChaCha20-Poly1305)                                          |

# Question

When your browser connects to a website over HTTPS, is the data protected by a block cipher or a stream cipher?



Encryption protects confidentiality

but how do we know the message was not altered, and that it really came from the right party?

# Message Authentication Code

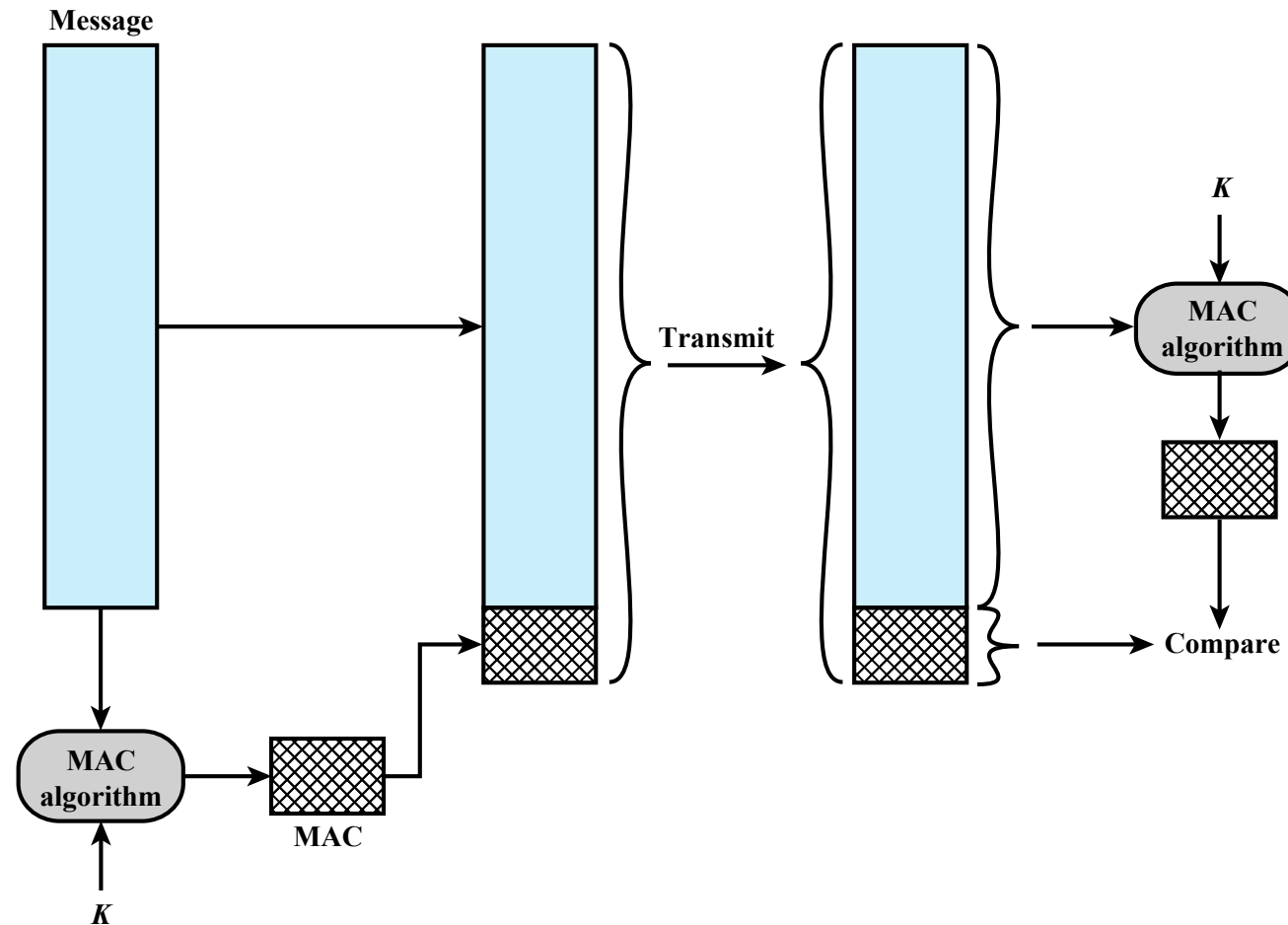


Figure 2.3 Message Authentication Using a Message Authentication Code (MAC).

# Two ways to generate MAC

- Use encryption algorithms
  - E.g., using DES to encrypt a message, retrieve last 16 bits or 32 bits of ciphertext as MAC (NIST FIPS 113)
- Use hash functions
  - A hash function accepts a variable-size message and outputs a fixed-size message digest
    - MD5, SHA-1, SHA-2, SHA-3

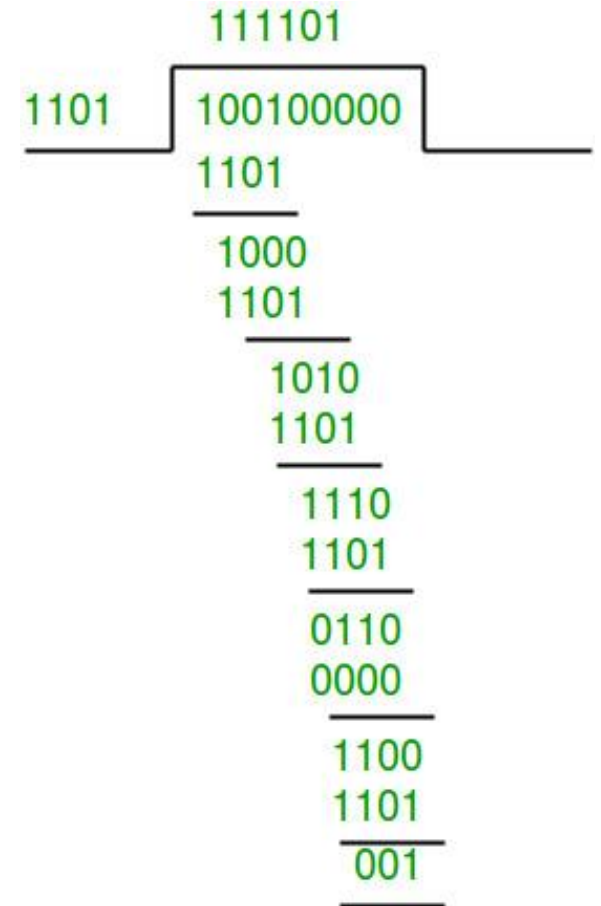
# Differences between CRC and MAC

## Key difference:

- CRC = "Did the channel corrupt this?"
- MAC = "Did an adversary tamper this, and did it come from the right party?"

## They work together, not instead of each other

- HTTPS packet: TLS MAC → TCP checksum → IP checksum → Ethernet CRC
- CRC drops noisy frames fast; MAC catches tampering above



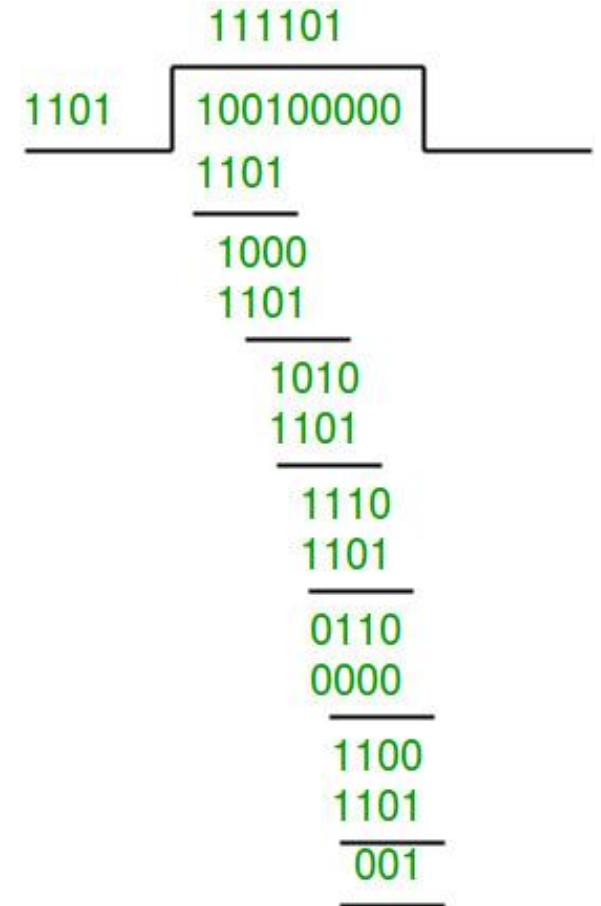
# Differences between CRC and MAC

## CRC (Cyclic Redundancy Check)

- Detects accidental errors: noise, bad cables, disk faults
- No key: anyone can compute it
- Lives at the low layers: Ethernet, Wi-Fi, USB, disk sectors

## MAC (Message Authentication Code)

- Detects intentional tampering: requires a secret key shared by sender and receiver
- Without the key, an attacker cannot produce a valid tag  
→ integrity + origin authentication
- Lives at the high layers: TLS, SSH, IPsec, AES-GCM tag

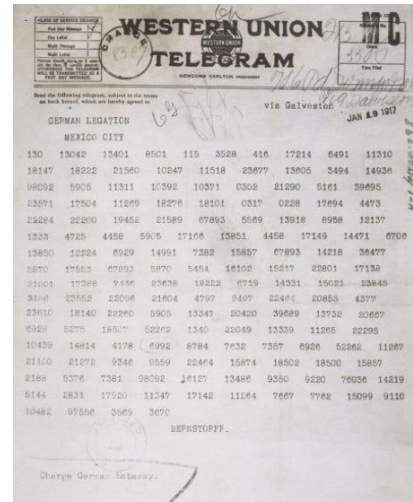




# Asymmetric Cryptography

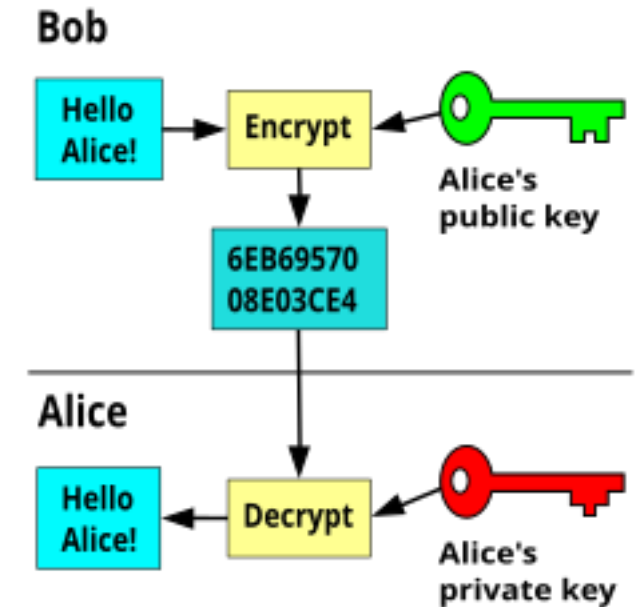
# Public-Key Cryptosystems

- Proposed by Diffie and Hellman in 1976
- Based on mathematical functions (e.g., modulo and exponential operations) rather simple operations on bit patterns
  - [Diffie-Hellman key agreement \(discrete logarithm problem\)](#),
  - RSA (prime factorization problem),
  - Elliptic Curve (elliptic curve discrete logarithm problem)



# Public-Key Cryptosystems

- Asymmetric
  - Use a key pair
    - A public key and a private key
  - Public key is open to the public
  - Private key is only known to the key pair owner
- If the ciphertext is encrypted by public key, it can be only decrypted by the corresponding private key
- If the ciphertext is encrypted by private key, it can be only decrypted by the corresponding public key



# Public-Key Cryptosystems

- Asymmetric
  - Use a key pair
    - A public key and a private key
  - Public key is open to the public
  - Private key is only known to the key pair owner

**Question:** Alice wants to send a secret message to Bob. Which key does she use?

**Alice encrypts the message with Bob's public key**

# Requirements for Public-Key Cryptography (Diffie & Hellman, 1976)

## For sender A and receiver B,

- It is computationally easy for a party A/B to generate a key pair (public key PU, private key PR)
- It is computationally easy for a sender A, knowing the public key and the message M, to generate the ciphertext:  $C = E(PU_b, M)$
- It is computationally easy for the receiver B to decrypt the ciphertext using the private key to recover the original message:  $M = D(PR_b, C)$

# Requirements for Public-Key Cryptography (Diffie & Hellman, 1976)

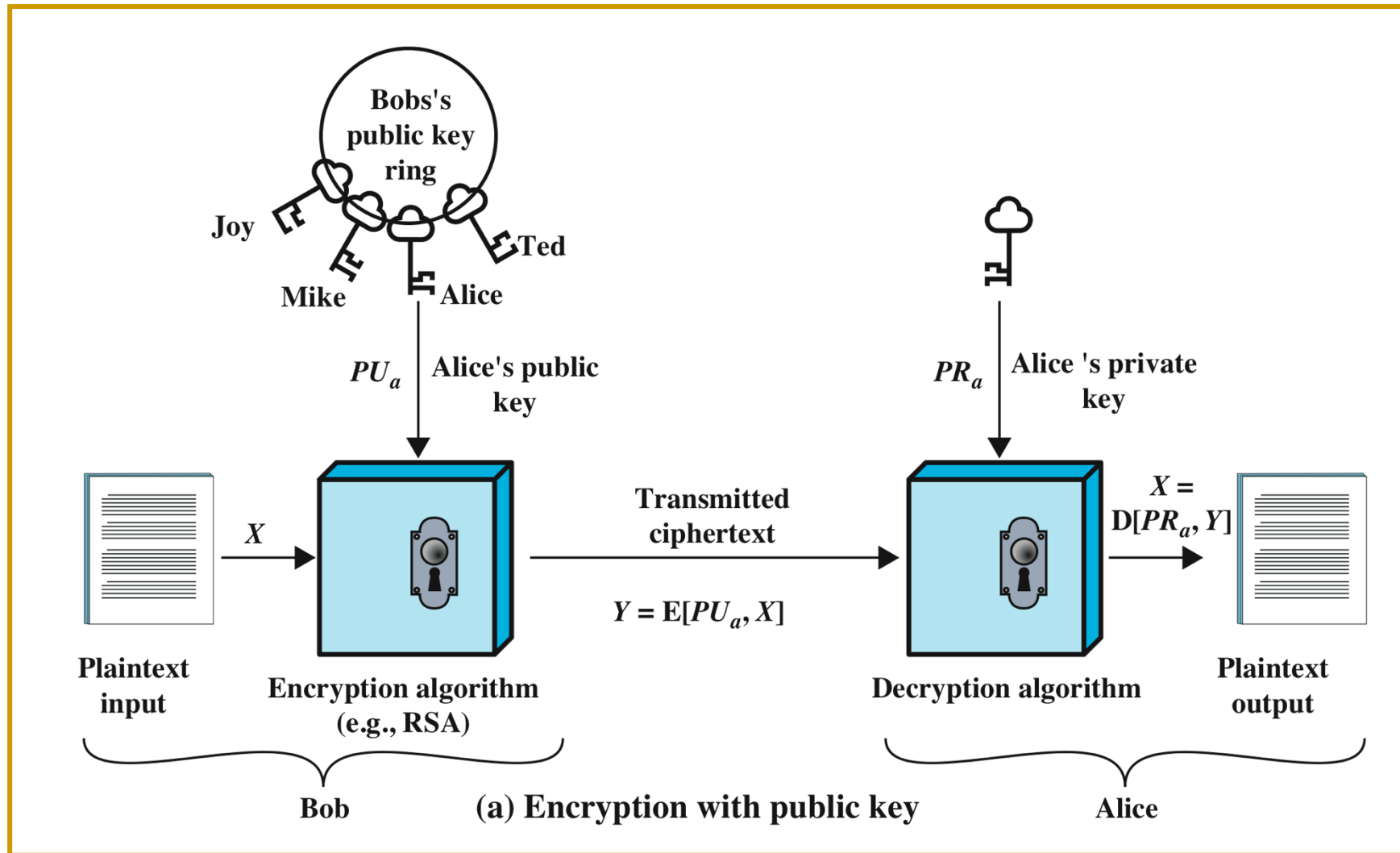
**For sender A and receiver B,**

- It is computationally infeasible for an adversary, knowing the public key  $PU_b$ , to determine the private key  $PR_b$
- It is computationally infeasible for an adversary, knowing the public key  $PU_b$  and a ciphertext  $C$ , to recover the original message  $M$

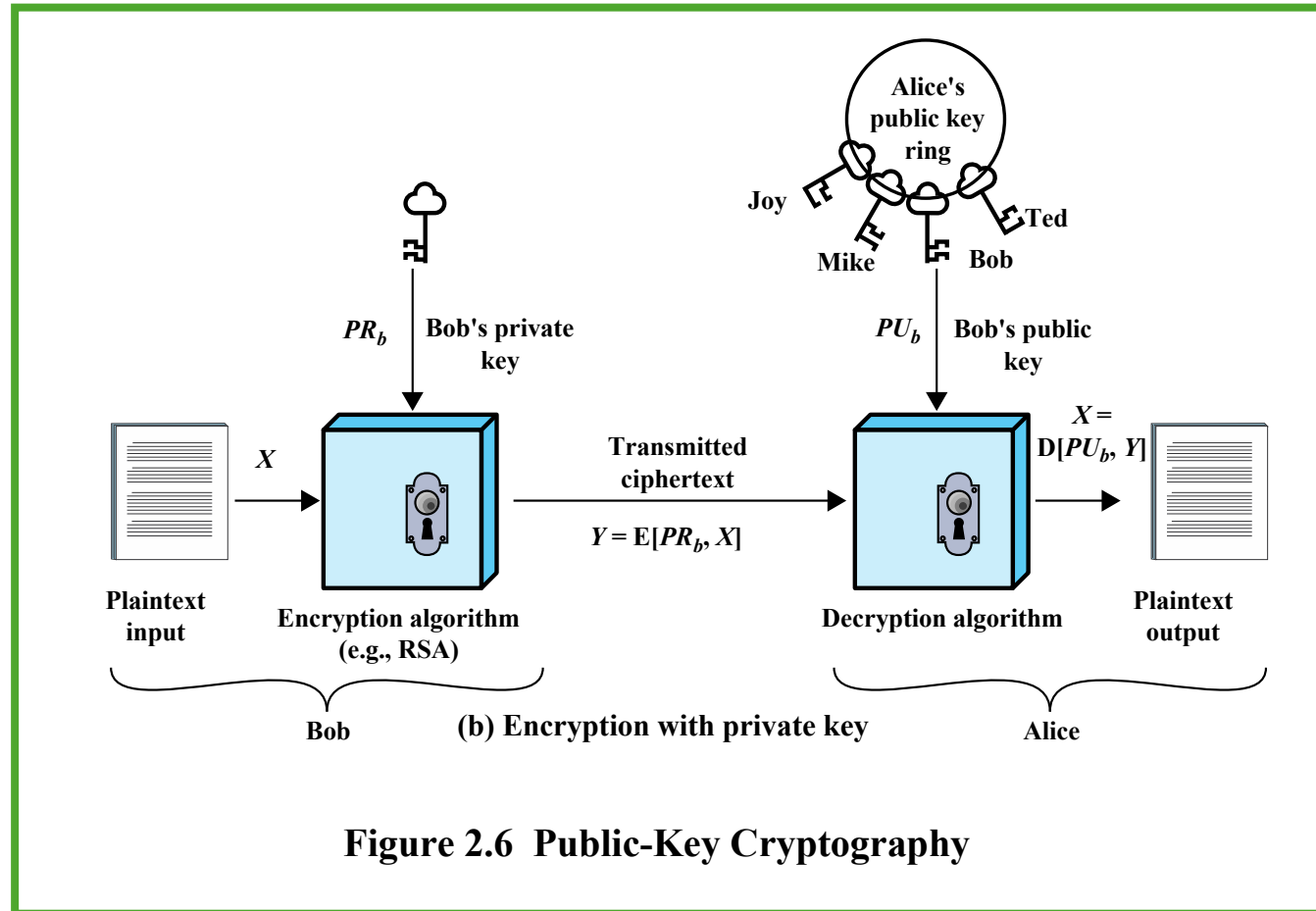
# Public-key cryptosystem applications

- Data confidentiality (encryption/decryption)
- Data integrity
- Key management (Key distribution)
- Digital signature

# Data confidentiality



# Data integrity



- Bob encrypts data using his or her own private key
- Anyone who knows the Bob's public key will be able to decrypt the message

# Why Quantum Computers Play Favorites

- **Shor's algorithm (1994):** solves prime factorization and discrete logarithms in polynomial time → breaks RSA and ECC completely, by exploiting their mathematical structure
- **Grover's algorithm (1996):** speeds up brute-force key search from  $2^k$  to  $2^{k/2}$  tries → only halves AES's effective key length, and a longer key compensates
- Shor can't touch AES: it has no algebraic structure to exploit; Grover applies to anything, but a quadratic speed-up is never fatal

**That's why the world is replacing RSA/ECC while simply keeping AES with longer keys**